

Copyright © 2026 Mihai SIMA

All rights reserved.

No part of the materials including graphics or logos, available in these notes may be copied, photocopied, reproduced, translated or reduced to any electronic medium or machine-readable form, in whole or in part, without specific permission. Distribution for commercial purposes is prohibited.

Disclaimer

The purpose of this course is to present general techniques and concepts for the analysis, design, and utilization of embedded systems. The requirements of any real embedded system can be intimately connected with the environment in which the embedded system is deployed. The presented design examples should not be used as the full design for any real embedded system.

Lesson 100: Project Pilot Lesson

- 1 Introduction
- 2 List of Projects
- 3 Report organization
- 4 Project goals
- 5 Project scenario
- 6 Application-specific ARM instructions
- 7 Firmware approach
- 8 Hardware solution
- 9 Project design flow – review
- 10 Understanding the algorithm
- 11 The complexity of the design task
- 12 Optimization Flags
- 13 Tools for Application-Level CPU Profiling
- 14 Installing the ARM virtual machine
- 15 ARM Single Board Computer

Introduction

- **Topic example:** implement Discrete Cosine Transform on an:
 - ARM-based system in which
 - the **butterfly** operation benefits from hardware/firmware support
- **Goal:** to practice navigating the hardware-software space
- What you practice for the project is also useful for the midterm and final exams
 - Software optimization techniques
 - Fixed-point arithmetic
 - Standard peripherals
- The text will be typed in 11- or 12-point type, single spaced
- The references will be listed at the end of the report
- The technical report must not exceed 20 pages

List of Projects

- 1 Convergence Computing Method (CCM)
- 2 COordinate Rotation Digital Computer (CORDIC)
- 3 Color Space Conversion (RGB-to-YCC and YCC-to-RGB)
- 4 Audio compression and decompression (A law, μ law)
- 5 Huffman encoding and decoding
- 6 Digital filters (FIR and IIR)
- 7 Motion estimation
- 8 RSA Cryptography based on Montgomery multiplication
- 9 Discrete Cosine Transform and Inverse Discrete Cosine Transform
- 10 Discrete Fourier Transform and Inverse Discrete Fourier Transform
- 11 Matrix inversion through Gauss-Jordan elimination
- 12 Matrix diagonalization (QR decomposition, eigenvalue decomposition, singular-value decomposition)
- 13 Side-Channel Attacks

Report organization

- 1 **Front page** with title, author, affiliation and contact information (UVic, Dept. of CS/ECE, student number, e-mail) and a blank dotted area to be filled in with the submission date at the time of submission
- 2 **Introduction** including a short description of the domain, performance requirements, an enumeration of the contributions, and project organization.
- 3 **Theoretical background**
- 4 One or more sections regarding **the design process** itself
- 5 **Performance/cost evaluation**
- 6 **Conclusions**
- 7 **Bibliography**

Project goal: Exposure to hardware-software co-design

- 1 Specification and modeling (specs, UML charts, blue-prints)
- 2 Pure-software implementation (C and assembly code)
- 3 *Profile* the resulting executable and determine the bottleneck
- 4 Augment the standard instruction set with a new instruction
- 5 Estimate the latency of this new instruction by:
 - Designing the computing unit that implements the newly defined instruction and determine the latency through simulation or measurement
 - *Educated guess* (the instructor will provide guidance)
- 6 Instantiate the new instruction (using intrinsic functions or inline assembly) and inspect the generated assembly listing
- 7 Build a testbench and test the system
- 8 Determine the speed-up / cost over the pure-software approach
- 9 Disseminate the information (technical report)

Result: **Application-Specific Instruction-set Processor (ASIP)**

Project goal: Fixed-Point Arithmetic

- Write C code that efficiently implements a given task in fixed-point arithmetic
- Fixed-point arithmetic requires a little bit of attention
 - Reduced dynamic range (example: with 8 bits we can represent at most 256 different values, so that the “entire world” is shrunk to 256 values)
 - Infinity cannot be mapped to a finite value, therefore it is the programmer responsibility to determine the largest value to be represented within a particular precision
- Truncation and round-off errors should be of particular concern
- Augment the ARM standard instruction set with a new instruction
 - Unfortunately, the compiler won't be able to detect the new instruction. Therefore, it is the programmer responsibility to explicitly instantiate the new operation (for example, by the use of *assembly inlining* constructs)

Project scenario – example (I)

Implement the 8×8 DCT on an ARM-based system in which the butterfly operation will benefit from hardware/firmware support

- 1 Describe the algorithm in a high-level language (C). The operation to benefit from hardware/firmware support will first be implemented as a routine, then as an *inline* routine, and finally as a macro.
- 2 Debug the high-level implementation by compiling it on a workstation. Profile-driven compilation may be needed. Also build the testbench (a set of input data with known output data).
- 3 Generate ARM assembly code for 8×8 DCT and *estimate* the performance of the pure-software solution. Specifically, determine the performance for all three butterfly software implementations: routine, *inline* routine, and macro.

Project scenario – example (II)

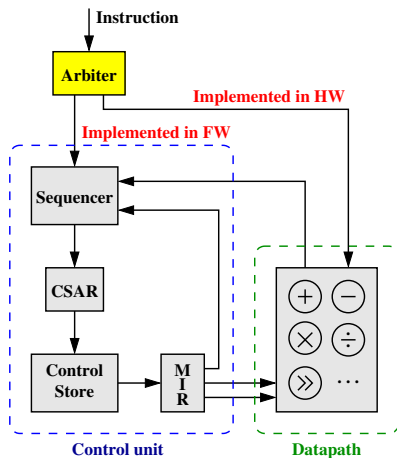
- 4 Use software optimization techniques (such as, software pipelining or loop unrolling) to improve the performance.
- 5 Optimize by hand the assembly code. Assemble the code and run the executable on an ARM machine (e.g., the virtual machine or the physical board connected to UVic intranet).
- 6 Instead of calling or inlining the butterfly routine, instantiate the butterfly operation. Only compile the code (as it is not possible to assemble it). To determine the latency of the butterfly operation, implement it in hardware or firmware.
- 7 Implement the butterfly in firmware. For the microcoded engine, use the ARM instruction set without multiplication and division. Investigate single-issue-slot and dual-issue-slot microcoded engines and report their cycle counts (computed manually, since no simulator is yet available for the microcoded engine).

Project scenario – example (III)

- 8 Implement the butterfly in hardware. Design an automaton with either D or J-K flip-flops. Also create the testbench (a set of inputs with known outputs). To determine the butterfly latency, you will compile the VHDL code and simulate the resulting circuit.
- 9 Describe the computing scenario, the organization, as well as the components of the system in UML, and provide the blue-prints of the design.
- 10 Upload all the files (C, assembly, VHDL/Verilog, UML, etc) and the presentation slides to Brightspace.
- 11 Upload the technical report to Brightspace.
- 12 Present your work and answer questions.

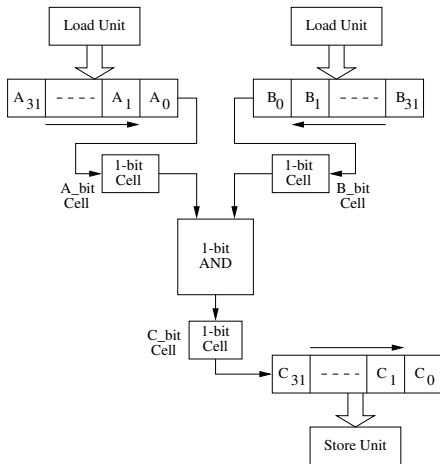
Firmware approach

- The application-specific instruction is executed as a microprogram
- Control Store is a 'non-standard' memory (e.g., in terms of bitwidth)
- In fact, the microinstruction set is 'non-standard'
- Vertical microcode – a single (micro)-computing unit is controlled per cycle (1 issue-slot)
- Horizontal microcode – multiple (micro)-computing units are controlled per cycle (2 issue-slots)



Vertical microcode – an example (I)

- Assume a processor whose instruction set includes a 32-bit bit-wise AND instruction
- Assume that the 32-bit bit-wise AND is a ‘complex’ instruction that is executed in microcode
- Consider an implementation that performs the AND operation serially, 1-bit per cycle



Vertical microcode – an example (II)

```

01          | LOAD  A
02          | LOAD  B
03  loop:   | SHIFT A
04          | SHIFT B
05          | AND    A_bit,B_bit,C_bit
06          | SHIFT C
07          | BACK   loop, 31_TIMES
08          | STORE C
09          | ENDOF

```

- 9 cycles are needed
- There are not many dependencies, thus horizontal microcode may provide for significant speed-up

Horizontal microcode (2 issue slots) – an example

01		LOAD	A		LOAD	B	
02	loop:		SHIFT	A		SHIFT	B
03		AND	A_bit,B_bit,C_bit		NO		
04		SHIFT	C		BACK	loop,31_TIMES	
05		STORE	C		ENDOP		

- 5 cycles are needed
- Speed-up of almost $2\times$

What to do for the 'firmware' section of project

- Write / generate assembly code for the bottleneck function
- Assume a dual-issue-slot microcode engine and fill out with (micro)-operations as many issue-slots as possible. This is the horizontal microcode
- Compare the 1 issue-slot solution with the 2 issue-slot solution
- How close are you to the maximum speed-up of $2\times$?
- Compare the 1 issue-slot solution with a 3 issue-slot solution
- How close are you to the maximum speed-up of $3\times$?

Hardware solution

- Zeroth-order system: combinational circuit
 - From register file back to register file in one step
 - It is not possible to issue a new instruction before the current instruction completes its processing
- First-order system: pipeline
 - From register file back to register file in multiple steps
 - It is now possible to issue a new instruction before the current instruction completes (in this case we have a pipelined processor)
 - Appropriate when large amount data is processed in a repetitive fashion
- Second-order system: automaton
 - Complex instructions can be implemented

ARM processor simulator – a simple C program

```
#include <stdio.h>

int main( int argc, char *argv[]) {
    int i, j, k=0;

    if ( argc != 2) {
        printf( "Enter exactly one argument\n");
        exit( 1); /* shell errors... */
    }

    i = atoi( argv[1]);
    for( j=0; j<i; j++)
        k += j;
    printf( "k = %i\n", k);
    exit( 0);
}
```

ARM processor simulator

- Boot the virtual machine and log in
- Compile with static libraries, since the ARM processor simulator might not support dynamic libraries

```
$ arm-linux-gnueabihf-gcc -static \  
    -march=armv5 file.c -o file.exe
```

- Simulate the resulting executable (file) with:

```
$ file.exe
```

- The output of the simulator

```
k = ...
```


The project design flow – review

- Assume that algorithm **A** is to be implemented
- Write high-level (C) code to implement algorithm **A**, then compile, simulate, and profile. Assume that function **B** is the bottleneck
- We now try to optimize function **B**:
 - We first try to inline the function **B**
 - We then optimize the assembly code we get by compilation
 - We finally write function **B** in assembly
- Assume that the optimized function **B** is still the bottleneck
- Define a new instruction **EXECUTE_B** and design a new computing unit that implements function **B**:
 - Firmware solution: the new computing unit is microcoded
 - Hardware solution: custom hardware is designed for a new computing unit

Instantiating machine-level custom instructions

- Assume the following C code:

```
int function_B( int a, int b) {
    <do something on a and b>
    return ...
}
```

```
int main( void) {
    int m, n, s;
    s = function_B( m, n);
}
```

- We want to support **function_B()** in hardware/firmware
- That is, we build new hardware/firmware and **extend** the machine instruction set with a new instruction: **EXECUTE B**

Instantiating machine-level custom instructions (cont'd)

- How to call the newly defined machine instruction from within C?
- By inlining assembly code:

```
int m, n, s;  
asm ("EXECUTE B %0,%1,%2" : "=r"(s) : "r"(m), "r"(n));
```

- Compile:

```
$ arm-none-gnueabihf-gcc -static \
    -march=armv5 -S file.c
```

- The resulting assembly code includes:

```
EXECUTE_B R1, R2, R3
```

- You cannot assemble such code, since the assembler cannot allocate an operation code for **EXECUTE_B**
- Augmenting the assembler with user-defined opcodes is beyond the project's scope

The complexity of the design task

■ Algorithm design

- Knowledge of the particular domain you are working on
- Mathematics
- Computer arithmetic

■ Software design

- Programming languages
- Optimization techniques
- Computer architecture and implementation

■ Hardware design

- Digital design
- Computer arithmetic
- Hardware description languages

Optimization Flags – C Code Example

```
#include <stdio.h>

volatile int a, b, c, d, e;

int f_01( int a, int b,
          int c, int d) {
    register int x, y;
    x = 0;
    if( c > 0)
        x = a + (b << 1);
    if( d < 0)
        y = (a >> 1) + b;
    else
        y = (a >> 1) - b;
    return x*y;
}
```

```
int main( void) {
    printf( "a = ");
    scanf( "%i", &a);
    printf( "b = ");
    scanf( "%i", &b);
    printf( "c = ");
    scanf( "%i", &c);
    printf( "d = ");
    scanf( "%i", &d);

    e = f_01( a, b, c, d);

    printf( "e = %i\n\n", e);
    return( 0);
}
```

```
arm-linux-gnueabihf-gcc -S test.c
```

The "-S" flag generates assembly code in a human readable format

```
f_01:    str    fp, [sp, #-4]!           .L2:    ldr    r3, [fp, #-20]
        add    fp, sp, #0              cmp    r3, #0
        sub    sp, sp, #28             bge    .L3
        str    r0, [fp, #-8]           ldr    r3, [fp, #-8]
        str    r1, [fp, #-12]          mov    r2, r3, asr #1
        str    r2, [fp, #-16]          ldr    r3, [fp, #-12]
        str    r3, [fp, #-20]          add    r3, r2, r3
        mov    r2, #0                  str    r3, [fp, #-24]
        str    r2, [fp, #-28]          b       .L4
        ldr    r3, [fp, #-16]          .L3:    ldr    r3, [fp, #-8]
        cmp    r3, #0                  mov    r2, r3, asr #1
        ble    .L2                    ldr    r3, [fp, #-12]
        ldr    r3, [fp, #-12]          rsb    r3, r3, r2
        mov    r2, r3, asl #1          str    r3, [fp, #-24]
        ldr    r3, [fp, #-8]          .L4:    ldr    r3, [fp, #-28]
        add    r3, r2, r3              ldr    r2, [fp, #-24]
        str    r3, [fp, #-28]          ...
```



```
arm-linux-gnueabihf-gcc -marm -O1 -S test.c
```

The "-m" flag with parameter "arm" forces the full ARM instruction set

The optimization flag `"-O"` with argument `"1"` enables the first optimization level

f_01:

```

mov      ip, r3
cmp      r2, #0
movle    r3, #0
movgt    r3, r1, asl #1
addgt    r3, r3, r0
cmp      ip, #0
addlt    r1, r1, r0, asr #1
rsbge    r1, r1, r0, asr #1
mul      r0, r3, r1
bx       lr

```

- Predicate operations are used

```
arm-linux-gnueabihf-gcc -mthumb -O1 -S test.c
```

The "-m" flag with parameter "thumb" forces the THUMB instruction set

The optimization flag "-O" with argument "1" enables the first optimization level

```
f_01:                                bge      .L4
    push    {r4, lr}                asr      r3, r0, #1
    mov     r4, r3                  add      r1, r3, r1
    cmp     r2, #0                  b        .L5
    bgt     .L2                    .L4:
    mov     r2, #0                  asr      r3, r0, #1
    b       .L3                    sub      r1, r3, r1
.L2:                                .L5:
    lsl     r3, r1, #1              mov      r0, r2
    add     r2, r3, r0              mul      r0, r1
.L3:                                pop      {r4}
    cmp     r4, #0                  pop      {r1}
                                bx         r1
```

```
arm-linux-gnueabihf-gcc -marm -Os -S test.c
```

The optimization flag "-O" with argument "s" minimizes the code size

```
f_01:
    cmp     r2, #0
    addgt   r2, r0, r1, asl #1
    movle   r2, #0
    mov     r0, r0, asr #1
    cmp     r3, #0
    addlt   r1, r0, r1
    rsbge   r1, r1, r0
    mul     r0, r2, r1
    bx      lr
```

```
arm-linux-gcc -mthumb -Os -S test.c
```

The optimization flag "-O" with argument "s" minimizes the code size

```
f_01:                                cmp     r4, #0
    push    {r4, lr}                bge     .L4
    mov     r4, r3                  add     r1, r0, r1
    cmp     r2, #0                  b       .L5
    bgt     .L2                    .L4:
    mov     r3, #0                  sub     r1, r0, r1
    b       .L3                    .L5:
.L2:                                mov     r0, r3
    lsl     r3, r1, #1              mul     r0, r1
    add     r3, r3, r0              pop     {r4}
.L3:                                pop     {r1}
    asr     r0, r0, #1              bx      r1
```

gcc

- Learning **gcc** is a good investment in your education
 - It is widely used in industry and academia
 - Lots of tutorials and documentation online
- Good textbooks:
 - Brian J. Gough, An Introduction to GCC, Network Theory Ltd., 2004.
 - Richard M. Stallman, Using the GNU Compiler Collection, GNU Press, multiple editions.
- You are highly recommended to become proficient with **gcc**

Tools for Application-Level CPU Profiling

- Many Linux tools to profile and instrument executables
- **gprof** requires profiling code to be included into the executable

```
$ gcc test.c -pg -o test.exe
```

```
$ ./test.exe (generates gmon.out)
```

```
$ gprof (reads gmon.out)
```

- **perf**, which is integrated into the Linux kernel

```
$ gcc test.c -o test.exe
```

```
$ perf record ./test.exe (generates perf.data)
```

```
$ perf report
```

- **valgrind** is a framework with many tools (next slides)

valgrind – CPU profiling tool

Multiple tools incorporated under the **valgrind** umbrella:

- **memcheck** = memory error detector
- **cachegrind** = cache and branch-prediction profiler
- **callgrind** = call-graph generating cache and branch prediction profiler
- **helgrind** = thread error detector
- **DRD** = thread error detector
- **massif** = heap profiler
- **DHAT** = dynamic heap analysis tool
- ... and many others

<http://www.valgrind.org/>

valgrind – Example (I)

- Consider the file `test.c`, which includes a function `function_1`
 - The instruction count of `function_1` needs to be minimized
 - The number of cache misses needs to be minimized
 - Branch statistics needs to be collected
-
- **valgrind** is non-invasive
 - Only the executable is needed for analysis

valgrind – Example (II)

- To determine the number of instructions in **function 1**:

```
$ gcc test.c -o test.exe
```

```
$ valgrind -tool=callgrind ./test.exe
```

```
$ callgrind_annotate callgrind.out.PID |  
grep function_1
```

```
76 ???:function_1 [/home/seng440/test.exe]
```

- To count the number of cache misses and also collect branch statistics:

```
$ gcc test.c -o test.exe
```

```
$ valgrind -tool=cachegrind  
-branch-sim=yes ./test.exe
```

Software Systems – Virtual Machines (I)

- Disk images of ARM-based virtual machines running Linux
 - They are minimal installations (no graphic interface)
 - Include development tools including **gcc** compiler (OS and bare-metal)
 - Can be downloaded from Engineering intranet
 - Installation instructions are provided in the appendix
- Each installation has been tested on:
 - Windows 11
 - OpenSUSE Leap
 - Centos 7
 - Ubuntu
 - macOS? ... any volunteer?
- **QEMU** is a very good instruction-set emulator
 - It runs on Linux, Windows, and macOS
 - It emulates ARM/AArch64, x86/x86-64, PowerPC, ...

Software Systems – Virtual Machines (II)

- **Virtual Manager** is a wrapper around QEMU
 - It runs on Linux, Windows, and macOS
 - Installation is not very user-friendly
- **Virtualbox** is a hypervisor
 - It virtualizes the host architecture
 - It runs on Windows, Linux, and macOS
- **VMware** is a hypervisor
 - It virtualizes the host architecture
 - It runs on Windows, Linux, and macOS
- The instructor prefers using QEMU from the command line (that is, without Virtual Manager)

Software Repositories (I)

- To download files, you must first establish a VPN connection, and then provide your NetLink ID and password
 - <https://software.engr.uvic.ca/seng/seng440/>
- The total size of the archive is about 20 GB
- Top directory includes five subdirectories:
 - **ARMHF32_VM** 32-bit ARM virtual machine
 - **ARM64_VM** 64-bit ARM virtual machine
 - **ARMHF32_KIT_debian** 32-bit ARM Linux kit
 - **ARM64_KIT_debian** 64-bit ARM Linux kit
 - **OEMU KIT For Windows 11** QEMU kit for Windows 11

Software Repositories (II)

- The 32-bit ARM virtual machine subdirectory includes:
 - `armhf32_VHD.qcow2`..... disk image
 - `vmlinuz_installed`..... Linux kernel executable
 - `initrd_installed.img`..... initial RAM disk
 - `ARMHF32_create_VHD_win.cmd`..... Windows script to create the virtual hard drive
 - `ARMHF32_install_win.cmd`..... Windows script to install the ARM Linux
 - `ARMHF32_run_wo_UEFI_win.cmd`..... launch the virtual machine under Windows 11
 - `ARMHF32_run_wo_UEFI_linux.sh`..... launch the virtual machine under Linux
 - `ARMHF32_config_script_linux.sh`..... configuration file for the previous script

ARM Single-Board Computer (I)

- If you encounter difficulties in installing the Virtual ARM Machine, you have the option of using the Real ARM Single-Board Computer (SBC) available within Engineering intranet
- For this real machine, a complete ARM toolchain is also available inside campus network. Just login on any lab machine in the ECE Undergraduate Lab using your netlink ID and password:
`$ ssh ugls.ece.uvic.ca`
- Your home directory is stored on a large harddrive, which has a capacity over 11 TB. This creates compatibility issues with not so new compilers (like `gcc` version `4.3.2`). It is recommended to create and use a temporary directory under the `/tmp` partition to generate the executables

ARM Single-Board Computer (II)

- It is your responsibility to save all your files under `/tmp` (for example, by copying them to your home directory), since this partition is erased every week
- The compiler is available under `/opt/arm/4.3.2/bin/` directory. To compile the source `example.c`, just type:

```
$ arm-linux-gcc -static -o example.exe example.c
```

- The compiler supports SIMD (NEON) intrinsics. Let us assume the following C code (notice that NEON header is included):

```
#include "arm_neon.h"
uint32x2_t aa, bb, ss;
int main( void) {
ss = vadd_u32( aa, bb);
}
```

ARM Single-Board Computer (III)

- To compile with NEON intrinsics, just type:

```
$ /opt/arm/4.3.2/bin/arm-linux-gcc\  
    -mfloat-abi=softfp\  
    -mfpu=neon -static -O3 -S example.c
```
- To access the ARM SBC, first **ssh** into **seng440.ece.uvic.ca** with your netlink ID/password, and then **telnet** into the ARM SBC:

```
$ telnet arm
```
- There are four users defined (user1, user2, user3, user4). The password for these four users is **q6coHjd7P**

Questions, feedback



Notes (I)

Notes (II)

Notes (III)